JAVA PLACEMENT MASTER SERIES · HANDBOOK 3

Collections Framework & Modern Java

HashMap internals, Streams, Optional & Java 8–21 essentials — the highest-priority handbook for coding rounds

★ HashMap Deep Dive

★ Streams & Collectors Mastery

★ Records & Pattern Matching

★ Top 50 + Top 30 Interview Q&A

For MCA / B.Tech / B.E. Placement Preparation

Coding Rounds · Java Backend Roles · Spring Boot Prep

Table of Contents

Tip: If you're short on time, revise **Part 22 (Final Revision)** and **Part 6 (HashMap internals)** first — HashMap and Streams are the two most-tested topics in Java coding interviews.

How to use this handbook

This is **Handbook 3** of the Java Placement Master Series — and the **highest-priority handbook** for placement preparation. It covers the Collections Framework and Java 8+ features that show up in nearly every coding round and technical interview: HashMap internals, Streams, Collectors, Optional, and modern Java (17/21) essentials. Read it top to bottom once, then use **Part 22 (Final Revision)** as your fast recap the night before an interview.

Importance tags used throughout:

- ⭐⭐⭐⭐⭐ Must Know for Placements
- ⭐⭐⭐⭐ Frequently Asked
- ⭐⭐ Medium Importance

Part 1 — Why Collections Exist ⭐⭐⭐⭐

Problems with Arrays

Arrays are **fixed-size** — once created, you can't grow or shrink them. Resizing means manually creating a new, bigger array and copying everything over. Arrays also only offer basic operations — no built-in searching by value, no easy insertion/removal in the middle, no ready-made Map/Set behavior.

```
int[] arr = new int[5];   // stuck at size 5 forever
// Need a 6th element? Must manually create a new int[6] and copy everything
```

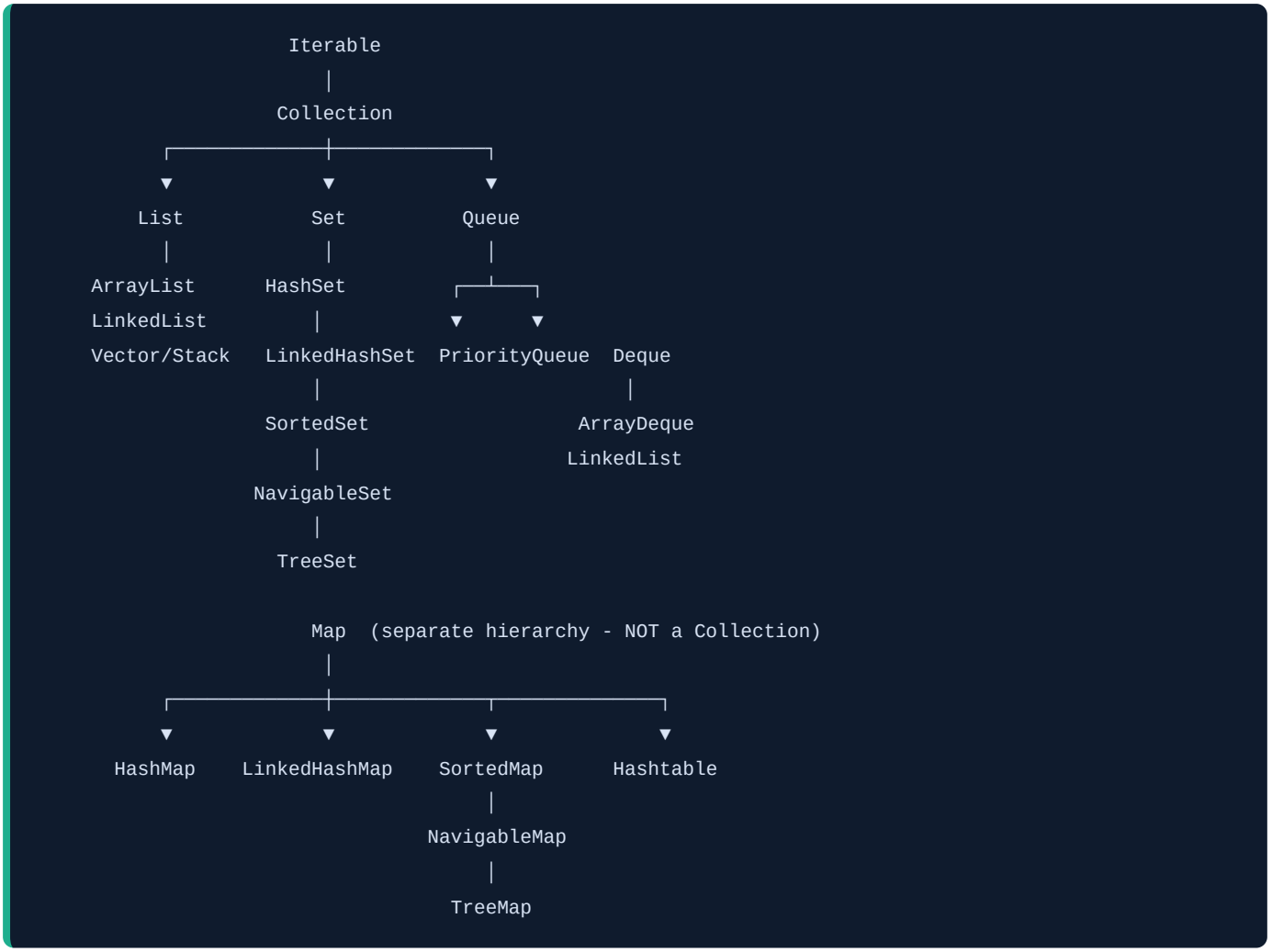
The Collections Framework

The **Collections Framework** is a unified architecture of interfaces and classes for storing and manipulating groups of objects — **resizable**, feature-rich, and interchangeable through common interfaces.

```
  Array  ──────►  Collection Framework
(fixed size,        │
 manual resize,     ├── List   (ordered, duplicates allowed)
 no built-in        ├── Set    (unique elements)
 search/sort)       ├── Queue  (FIFO / priority processing)
                    └── Map    (key-value pairs)
```

Benefits: dynamic resizing, ready-made algorithms (`sort` , `search` , `shuffle`), consistent APIs across implementations (swap `ArrayList` for `LinkedList` with minimal code change), and built-in thread-safe variants when needed.

Part 2 — Collection Hierarchy



Key fact ⭐⭐⭐⭐⭐: `Map` is **not** a subtype of `Collection` — it's a completely separate hierarchy, because a Map deals in key-value pairs, not single elements. This is a very common trick interview question.

Interface	Contract
`Iterable`	Can be looped over with a for-each loop (`Iterator<T> iterator()`)
`Collection`	Base contract for groups of objects: add, remove, size, contains
`List`	Ordered, indexed, allows duplicates
`Set`	No duplicates allowed
`Queue`	Designed for holding elements before processing (typically FIFO)
`Deque`	Double-ended queue — insert/remove from both ends
`SortedSet` / `SortedMap`	Maintains elements/keys in sorted order
`NavigableSet` / `NavigableMap`	Adds navigation methods: `floor()` , `ceiling()` , `higher()` , `lower()`

Part 3 — List Implementations ⭐⭐⭐⭐⭐

ArrayList ⭐⭐⭐⭐⭐

Backed internally by a **resizable array**. When it fills up, a new, bigger array is allocated (**growth factor: 1.5x** the current capacity) and all elements are copied over.

```
Capacity 10, full ⟶ new array of capacity 15 ⟶ copy all 10 elements over ⟶ add new one
```

Operation	Time Complexity	Why
`get(index)`	O(1)	Direct array index access
`add(element)` (at end)	O(1) amortized	Occasional resize costs O(n), but averages out
`add(index, element)` (middle)	O(n)	Must shift all subsequent elements
`remove(index)`	O(n)	Must shift elements to fill the gap
`contains(element)`	O(n)	Linear scan, no hashing

Interview trap ⭐⭐⭐⭐⭐: Removing while iterating with a for-each loop throws `ConcurrentModificationException`. Removing by **index** while iterating with a plain for-loop silently **skips elements** (indices shift after removal).

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4));
for (int i = 0; i < list.size(); i++) {
    if (list.get(i) == 2) list.remove(i);   // shifts elements - next element gets skipped!
}
```

LinkedList ⭐⭐⭐⭐⭐

A **doubly linked list** — each node holds data plus references to the **previous** and **next** nodes. No contiguous memory, no resizing needed.

```
null ⟵ [prev|data|next] ⟷ [prev|data|next] ⟷ [prev|data|next] ⟶ null
             Node 1               Node 2               Node 3
```

Operation	Time Complexity	Why
`get(index)`	O(n)	Must traverse from head or tail
`addFirst()` / `addLast()`	O(1)	Direct pointer update, no shifting
`add(index, element)` (middle)	O(n) to find + O(1) to insert	Traversal dominates the cost
`remove(element)`	O(n)	Must search first

Memory overhead: Each `LinkedList` node stores two extra references (prev/next) plus object header — noticeably more memory per element than `ArrayList`'s tightly packed array.

Vector and Stack (legacy)

- **`Vector`** — like `ArrayList`, but all methods are **synchronized** (thread-safe, but slower). Mostly legacy; use `ArrayList` + explicit synchronization or `CopyOnWriteArrayList` in modern code.
- **`Stack`** — extends `Vector`, provides `push()` / `pop()` / `peek()` for LIFO behavior. Modern code prefers `ArrayDeque` as a stack (faster, not legacy-synchronized).

Comparison Table

	ArrayList	LinkedList	Vector
Backing structure	Resizable array	Doubly linked nodes	Resizable array
Random access `get(i)`	O(1)	O(n)	O(1)
Insert/delete at ends	O(1) amortized (end only)	O(1)	O(1) amortized
Insert/delete in middle	O(n)	O(n) (traversal) + O(1) (link update)	O(n)
Thread-safe	No	No	**Yes** (synchronized)
Memory overhead	Low	Higher (node pointers)	Low
Best use case	Frequent random access/reads	Frequent insert/delete at ends (queues, stacks)	Legacy thread-safe needs

Interview Questions — Part 3

Q1. When should you use ArrayList vs LinkedList? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Use `ArrayList` when you need frequent random access (`get(i)`) — it's O(1). Use `LinkedList` when you need frequent insertion/removal at the beginning or end — it's O(1) there vs `ArrayList` 's O(n) shift cost. In practice, `ArrayList` is the default choice for most cases due to better cache locality.
- **Common mistake:** Assuming LinkedList is "generally faster" — it's only faster for specific end-insertion patterns, and its per-node memory overhead and poor cache locality often make it slower overall in practice.
- **Difficulty:** Basic

Q2. What is ArrayList's growth factor, and why does it matter? ⭐⭐⭐⭐

- **Ideal Answer:** When full, ArrayList grows to 1.5x its current capacity (allocates a new array, copies elements over). This amortizes the resize cost across many additions, giving O(1) *amortized* add performance despite occasional O(n) resize operations.
- **Difficulty:** Intermediate

Q3. Why does removing an element by index inside a for-loop sometimes skip elements? ⭐⭐⭐⭐⭐

- **Ideal Answer:** After removal, all subsequent elements shift left by one index, but the loop counter still increments — so the element that shifted into the just-removed index is never visited. Fix: iterate backwards, or use an `Iterator` 's `remove()` method.
- **Why asked:** A very common real-world bug — tests whether the candidate has actually hit this in practice.
- **Difficulty:** Intermediate

Part 4 — Set Implementations ⭐⭐⭐⭐⭐

HashSet ⭐⭐⭐⭐⭐

Internally backed by a `HashMap` — every element you add is stored as a **key** in that hidden map, with a shared dummy constant as the value. Uniqueness relies entirely on `hashCode()` + `equals()`.

```
HashSet.add("apple")  internally does:  hiddenMap.put("apple", PRESENT)
```

- **No guaranteed order** — iteration order can differ from insertion order and isn't stable.
- Duplicate check: compute `hashCode()` → find the bucket → compare with `equals()` against existing entries in that bucket.
- Since Java 8, buckets with many collisions (8+) convert from a linked list to a **red-black tree** — this is called **treeification**, and it bounds worst-case lookup to O(log n) instead of O(n).

LinkedHashSet

Same as `HashSet`, but maintains a **doubly linked list** through all entries to preserve **insertion order** during iteration — small extra memory cost for that guarantee.

TreeSet ⭐⭐⭐⭐⭐

Backed by a **Red-Black Tree** (a self-balancing binary search tree) — keeps elements in **sorted order** automatically (natural ordering or a custom `Comparator`).

```
          50
        /    \
      30      70
     /  \    /  \
   20   40  60   80
```

Provides **navigable** operations: `first()`, `last()`, `floor(x)`, `ceiling(x)`, `higher(x)`, `lower(x)`, `headSet()`, `tailSet()`.

Comparison Table

	HashSet	LinkedHashSet	TreeSet
Backing structure	HashMap	HashMap + linked list	Red-Black Tree
Order	No guarantee	Insertion order	Sorted order
`add` / `remove` / `contains`	O(1) average	O(1) average	O(log n)
Allows `null`	One `null` allowed	One `null` allowed	No (`NullPointerException`)
Best use case	Fast uniqueness check, order doesn't matter	Uniqueness + predictable iteration order	Uniqueness + sorted iteration/range queries

Interview Questions — Part 4

Q1. How does HashSet ensure uniqueness internally? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** HashSet is backed by a HashMap; each added element becomes a key with a constant dummy value. Uniqueness is enforced via `hashCode()` (to locate the bucket) and `equals()` (to check for an actual duplicate within that bucket).
- **Difficulty:** Intermediate

Q2. Why can't TreeSet contain null? ⭐⭐⭐⭐

- **Ideal Answer:** TreeSet must compare elements to maintain sorted order (via `compareTo()` or a `Comparator`), and comparing `null` to anything throws a `NullPointerException` — so TreeSet disallows null entirely.
- **Difficulty:** Intermediate

Part 5 — Queue & Deque ⭐⭐⭐⭐

Queue — FIFO Processing

```
Enqueue (add) ──►  [ A | B | C ]  ──► Dequeue (poll) removes from front (A first)
```

`Queue` is designed for holding elements prior to processing, typically **FIFO** (First-In-First-Out). Key methods come in two flavors: throwing (`add` , `remove` , `element`) and safe/non-throwing (`offer` , `poll` , `peek`).

PriorityQueue ⭐⭐⭐⭐

Not FIFO — orders elements by **priority** (natural ordering or a `Comparator`), backed internally by a **binary heap** (a complete binary tree stored in an array).

```
PriorityQueue<Integer> pq = new PriorityQueue<>();      // min-heap by default
pq.addAll(List.of(5, 1, 8, 3));
System.out.println(pq.poll());   // 1 - smallest first
```

Operation	Time Complexity
`offer()` / `add()`	O(log n)
`poll()` (remove head)	O(log n)
`peek()`	O(1)

Real use case: task scheduling, Dijkstra's/A* algorithms, "top-K" problems.

ArrayDeque ⭐⭐⭐⭐⭐

A resizable-array-based **double-ended queue** — insertion/removal from **both ends** in O(1). Faster than `LinkedList` for stack/queue use (better cache locality, no per-node object overhead) and is the **modern recommended choice** over both legacy `Stack` and `LinkedList` for these patterns.

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2);      // use as a STACK (LIFO)
stack.pop();                         // removes 2

Deque<Integer> queue = new ArrayDeque<>();
queue.offer(1); queue.offer(2);     // use as a QUEUE (FIFO)
queue.poll();                        // removes 1
```

Interview Questions — Part 5

Q1. Why is ArrayDeque preferred over Stack and LinkedList? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `Stack` extends the legacy, unnecessarily synchronized `Vector` (slower). `LinkedList` has per-node memory overhead and poor cache locality. `ArrayDeque` is array-backed, unsynchronized, and offers O(1) operations at both ends with better performance — it's the modern recommended implementation for both stack and queue use cases.
- **Difficulty:** Intermediate

Q2. How is a PriorityQueue implemented internally? ⭐⭐⭐⭐

- **Ideal Answer:** As a binary heap stored in an array — the root (index 0) is always the highest-priority (or smallest, for a min-heap) element. Insertions and removals maintain the heap property in O(log n) via "bubble up"/"bubble down" operations.
- **Difficulty:** Advanced

Part 6 — Map Implementations ⭐⭐⭐⭐⭐

(The single most important topic for Java interviews — expect deep-dive questions on HashMap internals.)

HashMap ⭐⭐⭐⭐⭐

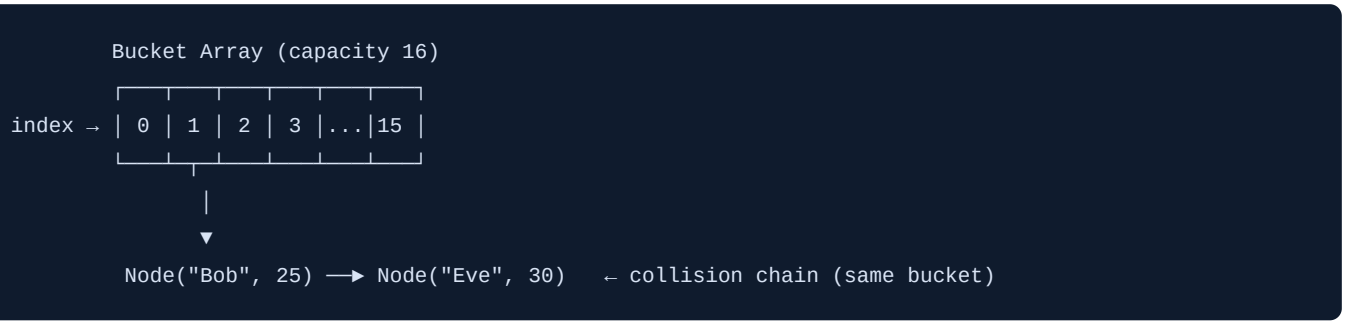
Node Structure

Internally, a `HashMap` is an **array of buckets**, where each bucket holds a linked list (or tree) of `Node` objects:

```
// Simplified internal Node structure
class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;    // points to the next node in the SAME bucket (collision chain)
}
```

How `put(key, value)` Works — Step by Step ⭐⭐⭐⭐⭐

```
1. Compute hash: hash(key.hashCode())     // extra bit-spreading for better distribution
2. Find bucket:  index = hash & (capacity - 1)     // equivalent to hash % capacity, faster
3. If bucket is empty → insert new Node directly
4. If bucket has entries (collision) →
      walk the chain, compare hash + equals() with each existing key
      if a match is found → update its value
      if no match → append a new Node to the end of the chain
```



Collision Handling & Treeification ⭐⭐⭐⭐⭐

When many keys hash into the **same bucket**, that bucket's chain grows long, degrading lookups toward O(n). Since **Java 8**, if a single bucket's chain grows to **8 or more nodes** (and the table has at least 64 buckets total), that bucket is converted from a linked list into a **red-black tree**, bounding worst-case lookup within that bucket to **O(log n)**. This is called **treeification**.

`equals()` and `hashCode()` — Why Both Matter ⭐⭐⭐⭐⭐

`hashCode()` picks the **bucket**; `equals()` confirms the **exact match** within that bucket. If you override `equals()` without `hashCode()` (or vice versa inconsistently), two "equal" objects can land in different buckets — breaking lookups and allowing duplicate keys to sneak in.

Rehashing and Load Factor ⭐⭐⭐⭐⭐

- **Load Factor** (default **0.75**) — when `size > capacity × loadFactor`, the map **resizes** (typically doubles capacity) and **rehashes** every existing entry into the new, larger bucket array.
- This 0.75 default balances memory usage against collision frequency — too low wastes memory, too high increases collisions.

```
capacity=16, loadFactor=0.75 → resize triggers at size=12
        |
        ▼
new capacity = 32, ALL existing entries rehashed into new bucket positions
```

Performance tip ⭐⭐⭐⭐: If you know the approximate final size upfront, pre-size the HashMap (`new HashMap<>(expectedSize)`) to avoid multiple expensive rehashing operations during population.

Null Handling & Iteration Order

- `HashMap` allows **one** `null` **key** and **multiple** `null` **values**.
- **Iteration order is not guaranteed** and can change after resizing — never rely on it.

Time Complexity

Operation	Average	Worst Case (pre-Java 8)	Worst Case (Java 8+, treeified)
`get` / `put` / `remove`	O(1)	O(n)	O(log n)

LinkedHashMap

Same mechanics as `HashMap`, plus a doubly linked list threading through all entries to preserve **insertion order** (or optionally, **access order** — useful for building an **LRU cache**).

```
// LRU cache using LinkedHashMap's access-order mode
LinkedHashMap<Integer, String> lru = new LinkedHashMap<>(16, 0.75f, true) {   // true = access order
    protected boolean removeEldestEntry(Map.Entry<Integer,String> eldest) {
        return size() > 100;    // evict oldest when capacity exceeded
    }
};
```

TreeMap ⭐⭐⭐⭐⭐

Backed by a **Red-Black Tree**, keeps keys in **sorted order**. Same navigable operations as `TreeSet`: `firstKey()`, `lastKey()`, `floorKey()`, `ceilingKey()`, `headMap()`, `tailMap()`. All operations are **O(log n)**.

Hashtable (legacy)

Like `HashMap`, but **synchronized** (thread-safe, slower) and **disallows** both null keys and null values entirely. Rarely used in modern code.

ConcurrentHashMap ⭐⭐⭐⭐

The modern, high-performance thread-safe alternative. Instead of locking the *entire* map (like `Hashtable`), it uses **fine-grained locking** (bucket/segment-level synchronization in older versions; per-node `synchronized` blocks + CAS operations in Java 8+), allowing many threads to read/write concurrently without blocking each other. Disallows `null` keys/values (to avoid ambiguity in concurrent reads).

Master Comparison Table

	HashMap	LinkedHashMap	TreeMap	Hashtable	ConcurrentHashMap
Order	None	Insertion (or access) order	Sorted	None	None
Null key/value	1 null key, many null values	Same as HashMap	No null key	No nulls at all	No nulls at all
Thread-safe	No	No	No	Yes (fully locked)	Yes (fine-grained)
Time complexity	O(1) avg	O(1) avg	O(log n)	O(1) avg (but slow, locked)	O(1) avg
Best use case	General-purpose default	Predictable iteration / LRU cache	Sorted keys, range queries	Legacy code only	Concurrent, multi-threaded access

Interview Questions — Part 6

Q1. Walk through what happens internally when you call `hashMap.put(key, value)`. ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** The key's `hashCode()` is computed and spread via an internal hash function, then mapped to a bucket index using `hash & (capacity-1)`. If the bucket is empty, a new node is inserted. If occupied (collision), the chain is walked comparing `hashCode` and `equals()`; a match updates the value, otherwise a new node is appended (or the bucket becomes a tree if it has 8+ entries).
- **Why asked:** THE most common Java collections deep-dive question — expected knowledge for any backend role.
- **Common mistake:** Vague answers like "it just stores it somewhere" without mentioning hashing, buckets, or collision handling.
- **Difficulty:** Advanced

Q2. What happens if you use a mutable object as a HashMap key and then change it after insertion? ⭐⭐⭐⭐⭐

- **Ideal Answer:** If the mutation changes the field(s) used in `hashCode()`, the object's hash changes, but it stays in its OLD bucket. Future lookups using an equal object compute a NEW hash pointing to a different bucket — so the entry becomes unreachable ("lost") even though it's technically still in the map.
- **Why asked:** Classic scenario question testing deep understanding, not just textbook definitions.
- **Difficulty:** Advanced

Q3. What is the default load factor, and why 0.75? ⭐⭐⭐⭐

- **Ideal Answer:** 0.75 is the default — a resize is triggered once the map is 75% full. It's a time-space tradeoff: a lower load factor wastes memory with mostly-empty buckets; a higher one increases collision chains and degrades lookup performance. 0.75 is a well-tested middle ground.
- **Difficulty:** Intermediate

Q4. HashMap vs Hashtable vs ConcurrentHashMap? ⭐⭐⭐⭐⭐

- **Ideal Answer:** HashMap is unsynchronized and allows one null key. Hashtable is fully synchronized (locks the whole map on every operation — slow) and disallows nulls entirely. ConcurrentHashMap is thread-safe via fine-grained locking (much better concurrent throughput than Hashtable) and also disallows nulls.
- **Difficulty:** Intermediate

Q5. What is treeification and why was it added in Java 8? ⭐⭐⭐⭐

- **Ideal Answer:** When a bucket's collision chain reaches 8+ nodes (with sufficient table capacity), HashMap converts that bucket from a linked list to a red-black tree, improving worst-case lookup from O(n) to O(log n) — protecting against performance collapse from poor hash distributions or hash-flooding attacks.
- **Difficulty:** Advanced

Part 7 — Comparable & Comparator ⭐⭐⭐⭐

	`Comparable`	`Comparator`
Defines	The class's own **natural ordering**	An **external, custom** ordering
Method	`compareTo(T other)`	`compare(T a, T b)`
Where implemented	Inside the class itself	A separate class/lambda, outside the class
How many orderings	Only one (the "natural" one)	Unlimited — define as many as needed

```
class Employee implements Comparable<Employee> {
    int salary;
    public int compareTo(Employee other) {
        return Integer.compare(this.salary, other.salary);   // natural order: by salary
    }
}
Collections.sort(employees);   // uses compareTo()

// Comparator - custom, external ordering, doesn't touch the Employee class
Comparator<Employee> byNameDesc = (a, b) -> b.name.compareTo(a.name);
employees.sort(byNameDesc);

// Modern lambda + method chaining, multi-level sorting
employees.sort(
    Comparator.comparing(Employee::getDept)
              .thenComparing(Employee::getSalary, Comparator.reverseOrder())
);
```

Interview Questions — Part 7

Q1. When would you use Comparator instead of Comparable? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Use `Comparable` for a class's single, obvious natural ordering (e.g., numbers sort by value). Use `Comparator` when you need multiple different orderings, or when you can't modify the class's source (e.g., it's from a third-party library).
- **Difficulty:** Basic

Part 8 — Iterator Family ⭐⭐⭐⭐

Iterator vs ListIterator

	`Iterator`	`ListIterator`
Direction	Forward only	**Both** forward and backward
Applicable to	Any `Collection`	Only `List`
Can modify during iteration	`remove()` only	`remove()`, `set()`, and `add()`

```
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int val = it.next();
    if (val == 3) it.remove();     // SAFE removal during iteration
}
```

Fail-Fast vs Fail-Safe ⭐⭐⭐⭐⭐

- **Fail-fast** (e.g., `ArrayList`, `HashMap` iterators) — detects **structural modification** (add/remove) during iteration via an internal `modCount` counter, and throws `ConcurrentModificationException` immediately.
- **Fail-safe** (e.g., `CopyOnWriteArrayList`, `ConcurrentHashMap` iterators) — iterates over a **snapshot** or tolerates concurrent structural changes without throwing, though the iteration may not reflect the very latest state.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
for (int val : list) {
    if (val == 2) list.remove(Integer.valueOf(2));   // ConcurrentModificationException!
}
```

Interview Questions — Part 8

Q1. What causes ConcurrentModificationException and how do you avoid it? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** It's thrown when a collection is structurally modified (add/remove) while being iterated with a fail-fast iterator, detected via an internal `modCount` mismatch. Avoid it by using `Iterator.remove()`, `ListIterator`, or a `CopyOnWriteArrayList` / `ConcurrentHashMap` for concurrent scenarios.
- **Difficulty:** Intermediate

Part 9 — Collections Utility Class ⭐⭐⭐⭐

`java.util.Collections` provides static helper methods for any `Collection`.

Method	Purpose
`Collections.sort(list)`	Sorts in place (natural order or with a Comparator)
`Collections.reverse(list)`	Reverses element order
`Collections.shuffle(list)`	Randomizes order
`Collections.frequency(list, x)`	Counts occurrences of `x`
`Collections.binarySearch(list, key)`	Fast search — **list must be sorted first**
`Collections.min(list)` / `max(list)`	Finds smallest/largest element
`Collections.synchronizedList(list)`	Wraps a collection to make it thread-safe (coarse locking)
`Collections.unmodifiableList(list)`	Returns a read-only **view** — mutation attempts throw `UnsupportedOperationException`
`Collections.emptyList()`	Returns an immutable empty list singleton

```
List<Integer> list = new ArrayList<>(List.of(5, 1, 4));
Collections.sort(list);                                  // [1, 4, 5]
List<Integer> readOnly = Collections.unmodifiableList(list);
// readOnly.add(9);   // throws UnsupportedOperationException
```

> ***Collection vs Collections*** ⭐⭐⭐⭐⭐**:** *`Collection` is an **interface** (the root of List/Set/Queue). `Collections` is a **utility class** full of static helper methods. Extremely common naming-confusion interview question.*

Part 10 — Java 8 Functional Programming ⭐⭐⭐⭐⭐

Core Functional Interfaces

A **functional interface** has exactly one abstract method, making it usable as a lambda target. `java.util.function` provides ready-made ones so you rarely need to write your own.

Interface	Signature	Purpose	Example
`Predicate<T>`	`boolean test(T t)`	A yes/no test	`n -> n % 2 == 0`
`Function<T,R>`	`R apply(T t)`	Transforms input to output	`s -> s.length()`
`Consumer<T>`	`void accept(T t)`	Takes input, returns nothing (side effect)	`s -> System.out.println(s)`
`Supplier<T>`	`T get()`	Takes nothing, produces a value	`() -> new ArrayList<>()`
`BiFunction<T,U,R>`	`R apply(T t, U u)`	Two inputs, one output	`(a,b) -> a + b`
`UnaryOperator<T>`	`T apply(T t)`	`Function<T,T>` — same input/output type	`n -> n * 2`
`BinaryOperator<T>`	`T apply(T t1, T t2)`	`BiFunction<T,T,T>` — combines two of the same type	`(a,b) -> a > b ? a : b`

```
Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4));           // true

Function<String, Integer> length = String::length;
System.out.println(length.apply("hello"));      // 5

Consumer<String> printer = System.out::println;
printer.accept("Hi!");

Supplier<Double> random = Math::random;
System.out.println(random.get());
```

Part 11 — Lambda Expressions ★★★★★

Syntax

```
(parameters) -> expression
(parameters) -> { statements; }

() -> 42                              // no parameters
x -> x * 2                            // one parameter, no parentheses needed
(x, y) -> x + y                       // multiple parameters
(String s) -> s.toUpperCase()         // explicit type (optional, usually inferred)
```

Capturing Variables — "Effectively Final" ★★★★★

A lambda can use a local variable from its enclosing scope **only if that variable is never reassigned** after initialization ("effectively final") — this is enforced by the compiler.

```
int factor = 10;
Function<Integer, Integer> multiply = n -> n * factor;   // OK - 'factor' never reassigned

int counter = 0;
Runnable r = () -> counter++;    // COMPILE ERROR - counter is reassigned, not effectively final
```

Why this restriction exists: Lambdas may outlive the method they were created in (e.g., passed to another thread). Java captures local variables **by value** (a copy), so if the original could still change, the lambda's copy would silently go stale — Java prevents this entire class of bug by disallowing it at compile time.

Under the Hood

A lambda is **not** simply syntactic sugar for an anonymous class — the JVM compiles it using `invokedynamic` and generates the implementation lazily at runtime (via `LambdaMetafactory`), which is more efficient (avoids a new `.class` file per lambda and reduces JAR bloat).

Part 12 — Method References ⭐⭐⭐⭐

A shorthand for a lambda that does nothing but **call an existing method**.

Type	Syntax	Equivalent Lambda
Static method	`ClassName::staticMethod`	`x -> ClassName.staticMethod(x)`
Instance method of a particular object	`object::instanceMethod`	`x -> object.instanceMethod(x)`
Instance method of an arbitrary object (of a type)	`ClassName::instanceMethod`	`(obj, x) -> obj.instanceMethod(x)`
Constructor reference	`ClassName::new`	`x -> new ClassName(x)`

```
Function<String, Integer> parse = Integer::parseInt;             // static method
Consumer<String> print = System.out::println;                     // instance method of particular
object
Function<String, Integer> len = String::length;                   // instance method of arbitrary
object
Supplier<ArrayList<String>> factory = ArrayList::new;               // constructor reference
```

Interview Questions — Part 10–12

Q1. What makes an interface a "functional interface"? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Having exactly one abstract method (it can have any number of default/static methods). The `@FunctionalInterface` annotation is optional but documents intent and lets the compiler catch accidental violations.
- **Difficulty:** Basic

Q2. What does "effectively final" mean and why is it required for lambda variable capture? ⭐⭐⭐⭐⭐

- **Ideal Answer:** A local variable that is assigned once and never reassigned afterward. Lambdas capture variables by value; if the original variable could change after the lambda captured it, the lambda would hold a stale copy — Java's compiler prevents this entire bug class by requiring effective finality.

- **Why asked:** A very popular "gotcha" question that trips up many candidates who don't understand the reasoning behind it.
- **Difficulty:** Advanced

Q3. Difference between Function and BiFunction? ⭐⭐⭐

- **Ideal Answer:** `Function<T,R>` takes one argument; `BiFunction<T,U,R>` takes two arguments (possibly different types) and produces one result.
- **Difficulty:** Basic

Part 13 — Streams API ⭐⭐⭐⭐⭐

What a Stream Is

A **Stream** is a pipeline for processing a sequence of elements in a **functional, declarative** style — "what to do," not "how to loop." Streams **don't store data** — they process data from a source (a collection, array, etc.) on demand.

```
  Source        Intermediate Ops (lazy)        Terminal Op (triggers execution)
List<Integer> ──► filter() ──► map() ──► sorted() ──► collect()/forEach()/reduce()
```

Lazy Evaluation ⭐⭐⭐⭐⭐

Intermediate operations (`filter`, `map`, `sorted`...) are **lazy** — they don't actually run until a **terminal operation** (`collect`, `forEach`, `reduce`...) is invoked. This allows the JVM to **fuse operations together** and process each element through the entire pipeline in one pass, rather than materializing an intermediate list after every step.

```
List<String> names = List.of("Alice", "Bob", "Charlie", "Dave");

List<String> result = names.stream()
    .filter(n -> { System.out.println("filter: " + n); return n.length() > 3; })
    .map(n -> { System.out.println("map: " + n); return n.toUpperCase(); })
    .collect(Collectors.toList());
// Notice the output interleaves filter/map PER ELEMENT, not filter-all-then-map-all!
```

Intermediate vs Terminal Operations

Intermediate (lazy, returns a Stream)	Terminal (eager, triggers execution)
`map`, `filter`, `flatMap`	`collect`, `forEach`, `reduce`
`distinct`, `sorted`, `peek`	`count`, `findFirst`, `findAny`
`limit`, `skip`	`anyMatch`, `allMatch`, `noneMatch`

Common Operations Reference

```
List<Integer> nums = List.of(5, 3, 8, 3, 1, 9, 2);

nums.stream().map(n -> n * 2)                    // transform each element
nums.stream().filter(n -> n > 3)                  // keep matching elements
nums.stream().distinct()                          // remove duplicates
nums.stream().sorted()                            // natural order sort
nums.stream().limit(3)                            // first 3 elements
nums.stream().skip(2)                             // skip first 2 elements
nums.stream().peek(System.out::println)           // debug - view without consuming

List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));
nested.stream().flatMap(List::stream)                  // flattens nested streams into one

nums.stream().forEach(System.out::println);            // terminal - consumes each element
long count = nums.stream().filter(n -> n > 3).count();    // terminal - counts matches
Optional<Integer> first = nums.stream().findFirst();        // terminal - first element
boolean any = nums.stream().anyMatch(n -> n > 8);             // short-circuits on first match
boolean all = nums.stream().allMatch(n -> n > 0);              // short-circuits on first failure
boolean none = nums.stream().noneMatch(n -> n > 100);
int sum = nums.stream().reduce(0, Integer::sum);                // combines all into one value
```

Short-Circuiting ⭐⭐⭐⭐

Operations like `limit`, `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch` can **stop processing early** without touching the entire source — critical for performance on large or infinite streams.

```
Stream.iterate(1, n -> n + 1)     // infinite stream!
      .filter(n -> n % 7 == 0)
      .findFirst();               // stops as soon as the first match (7) is found - doesn't hang
forever
```

`map` vs `flatMap` ⭐⭐⭐⭐⭐

```
map:      [ [1,2], [3,4] ]  ──►  [ Stream<[1,2]>, Stream<[3,4]> ]   (stream of streams - nested!)
flatMap:  [ [1,2], [3,4] ]  ──►  [ 1, 2, 3, 4 ]                      (flattened into one stream)
```

Use `map` for a 1-to-1 transformation. Use `flatMap` when each element itself produces a **stream/collection**, and you want a single, flattened result stream.

Interview Questions — Part 13

Q1. What is lazy evaluation in Streams, and why does it matter? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Intermediate operations build up a pipeline description but don't execute until a terminal operation is called. This lets the JVM fuse all operations and process each element through the whole pipeline in a single pass, and allows short-circuiting (e.g., `findFirst`) to avoid unnecessary work — critical for performance, especially on large or infinite streams.
- **Difficulty:** Intermediate

Q2. Difference between map and flatMap? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `map` transforms each element 1-to-1, potentially producing nested structures (a stream of streams). `flatMap` transforms each element into a stream and then flattens all those streams into a single, combined stream — used when processing nested collections.
- **Difficulty:** Intermediate

Q3. Can a Stream be reused/consumed twice? ⭐⭐⭐⭐

- **Ideal Answer:** No — a Stream can only be consumed by one terminal operation. Attempting to reuse a already-consumed stream throws `IllegalStateException`. A new stream must be created from the source each time.
- **Why asked:** A common runtime-error scenario in real code.
- **Difficulty:** Intermediate

Q4. What's the difference between forEach and peek? ⭐⭐⭐⭐

- **Ideal Answer:** `forEach` is a terminal operation, intended as the final consuming step of a pipeline. `peek` is an intermediate operation meant for debugging/side-effect inspection mid-pipeline — it shouldn't be relied on to do the "real work," and (being lazy) it may not execute at all if the pipeline short-circuits before reaching that element.
- **Difficulty:** Advanced

Part 14 — Collectors ⭐⭐⭐⭐⭐

`Collectors` (used with the terminal `.collect()` operation) turns a stream into a final result — a collection, a map, a summary value, or a joined string.

```
List<String> names = List.of("Amit", "Sara", "Amit", "Priya");

List<String> list = names.stream().collect(Collectors.toList());
Set<String> set = names.stream().collect(Collectors.toSet());

Map<String, Integer> nameLengths = names.stream()
    .distinct()
    .collect(Collectors.toMap(n -> n, String::length));    // key mapper, value mapper

String joined = names.stream().collect(Collectors.joining(", "));   // "Amit, Sara, Amit, Priya"

long count = names.stream().collect(Collectors.counting());
```

`groupingBy` and `partitioningBy` ⭐⭐⭐⭐⭐

```
List<String> words = List.of("cat", "dog", "cow", "duck", "ant");

// groupingBy - splits into MULTIPLE groups based on a classifier function
Map<Integer, List<String>> byLength = words.stream()
    .collect(Collectors.groupingBy(String::length));
// {3=[cat, dog, cow], 4=[duck, ant... wait "ant" has length 3]}
// -> {3=[cat, dog, cow, ant], 4=[duck]}

// partitioningBy - splits into EXACTLY two groups: true/false
Map<Boolean, List<String>> partitioned = words.stream()
    .collect(Collectors.partitioningBy(w -> w.length() > 3));
// {false=[cat, dog, cow, ant], true=[duck]}

// groupingBy + downstream collector
Map<Integer, Long> countByLength = words.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));
```

Other Useful Collectors

Collector	Purpose
`mapping()`	Applies a transformation before collecting (used as a downstream collector)
`collectingAndThen()`	Applies a final wrapping transformation after collecting (e.g., make the result immutable)
`summarizingInt/Long/Double()`	Produces count, sum, min, max, average in one pass

```
List<String> immutableList = names.stream()
    .collect(Collectors.collectingAndThen(Collectors.toList(), Collections::unmodifiableList));
```

Interview Questions — Part 14

Q1. groupingBy vs partitioningBy? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** `groupingBy` splits elements into an arbitrary number of groups based on a classifier function's return value (the map's keys). `partitioningBy` is a specialized case that always splits into exactly two groups — `true` and `false` — based on a predicate.
- **Difficulty:** Intermediate

Q2. How would you count word frequency in a list of strings? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `words.stream().collect(Collectors.groupingBy(w -> w, Collectors.counting()))` — groups identical words together and counts occurrences in each group in a single pipeline.
- **Why asked:** A very common coding-round Streams question.
- **Difficulty:** Intermediate

Part 15 — Optional ⭐⭐⭐⭐⭐

Why Optional Exists

Before `Optional`, a method returning "maybe nothing" (e.g., a lookup that might not find a result) simply returned `null` — and callers routinely forgot to check, causing `NullPointerException`. `Optional<T>` makes the **possibility of absence explicit in the type system** — the compiler and the API itself force you to consciously handle the "empty" case.

Creating and Checking

```
Optional<String> present = Optional.of("Hello");         // throws NPE if the argument is null
Optional<String> maybeNull = Optional.ofNullable(getValue());  // safely wraps a possibly-null value
Optional<String> empty = Optional.empty();

if (present.isPresent()) { System.out.println(present.get()); }   // old style - avoid where possible
present.ifPresent(val -> System.out.println(val));                 // preferred - functional style
```

Extracting Values Safely

```
String result1 = maybeNull.orElse("default");                    // fallback value (ALWAYS evaluated!)
String result2 = maybeNull.orElseGet(() -> computeDefault());      // fallback computed LAZILY, only
if needed
String result3 = maybeNull.orElseThrow(() -> new NoSuchElementException("not found"));
```

`orElse` vs `orElseGet` ⭐⭐⭐⭐⭐: `orElse(x)` always evaluates `x` eagerly — even if the Optional is present and `x` is never used! `orElseGet(supplier)` only invokes the supplier if the Optional is actually empty. If the default value is expensive to compute (a DB call, heavy computation), always prefer `orElseGet`.

```
// BAD - expensiveDefault() runs EVERY TIME, even when 'value' is present!
value.orElse(expensiveDefault());

// GOOD - only computed if 'value' is actually empty
value.orElseGet(() -> expensiveDefault());
```

`map` and `flatMap` on Optional

```
Optional<String> name = Optional.of("java");
Optional<Integer> length = name.map(String::length);     // Optional[4] - transforms if present, stays
empty if not

Optional<Optional<String>> nested = name.map(n -> Optional.of(n.toUpperCase()));   // nested -
awkward!
Optional<String> flat = name.flatMap(n -> Optional.of(n.toUpperCase()));            // flattened -
clean
```

Common Misuse ⭐⭐⭐⭐⭐

Mistake	Why It's Wrong
Calling `.get()` without checking `.isPresent()` first	Defeats the entire purpose — can still throw `NoSuchElementException`
Using `Optional` as a field or method parameter type	`Optional` is meant only as a **return type** signaling "may be absent" — not for general-purpose null-avoidance everywhere
Using `Optional<List<T>>`	Just return an **empty list** instead — avoids a pointless extra wrapping layer
Overusing `orElse()` with expensive computations	Always evaluates eagerly — use `orElseGet()` instead

Interview Questions — Part 15

Q1. Why was Optional introduced, and how should it be used? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** To make "a value might be absent" explicit in the type system, reducing accidental NullPointerExceptions. It should primarily be used as a **method return type** for lookups that might not find a result, encouraging callers to explicitly handle the empty case via `map`, `orElse`, `ifPresent`, etc., rather than calling `.get()` blindly.
- **Difficulty:** Basic

Q2. Difference between orElse and orElseGet? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `orElse(value)` always evaluates its argument eagerly, even when the Optional has a value present. `orElseGet(supplier)` only invokes the supplier lazily, when the Optional is actually empty — important when the default is expensive to compute.
- **Why asked:** A classic performance-awareness "gotcha" question.
- **Difficulty:** Intermediate

Q3. Why shouldn't Optional be used as a field type or method parameter? ⭐⭐⭐⭐

- **Ideal Answer:** Optional wasn't designed for that purpose (it's not `Serializable`, adds unnecessary wrapping overhead, and complicates object construction) — its intended use is exclusively as a return type to signal a possibly-absent result to a caller.
- **Difficulty:** Advanced

Part 16 — Modern Java Essentials ⭐⭐⭐⭐

Feature	Version	What It Does
`var`	Java 10	Local variable type inference — type fixed at compile time
Switch Expressions	Java 14	`switch` that returns a value, `->` syntax, no fall-through
Text Blocks	Java 15	Multi-line string literals with `"""` — no more `\n` concatenation
Pattern Matching for `instanceof`	Java 16	Auto-casts within the `if` — `if (obj instanceof String s)`
Records ⭐⭐⭐⭐⭐	Java 16	Concise immutable data classes — auto-generates constructor/getters/equals/hashCode/toString
Sealed Classes ⭐⭐⭐⭐	Java 17	Restricts which classes can extend/implement a type via `permits`
Pattern Matching for `switch`	Java 21	`switch` on an object's type, with destructuring support

```
// Text block (Java 15+)
String json = """
    {
      "name": "Java",
      "version": 21
    }
    """;

// Record (Java 16+)
record Point(int x, int y) {}
Point p = new Point(3, 4);
System.out.println(p.x() + ", " + p.y());   // accessor methods, not getX()/getY()

// Pattern matching for switch (Java 21)
Object obj = 42;
String desc = switch (obj) {
    case Integer i when i > 0 -> "Positive int: " + i;
    case String s             -> "String: " + s;
    default                    -> "Unknown";
};
```

Industry usage ⭐⭐⭐⭐⭐: Records are now the default choice for DTOs in modern Spring Boot apps (replacing verbose Lombok-generated or hand-written POJOs). Pattern matching + sealed classes together enable safer, more expressive domain modeling — increasingly expected knowledge as companies adopt Java 17/21 LTS.

Part 17 — Performance & Best Practices

⭐⭐⭐⭐

Practice	Why It Matters
Pre-size ArrayList/HashMap (`new ArrayList<>(1000)`)	Avoids repeated resize/copy or resize/rehash operations when the approximate final size is known
Avoid String concatenation in loops	Each `+` creates a new String object — use `StringBuilder` instead
Stream vs loop performance	For simple operations on small-to-medium data, a plain loop is often faster (less overhead); Streams shine for readability and for parallelization on large datasets
Boxing overhead	Autoboxing in tight loops (`List<Integer>` vs `int[]`) creates unnecessary objects — prefer primitive arrays/streams (`IntStream`) in performance-critical numeric code
Parallel stream caution ⭐⭐⭐⭐	`.parallelStream()` uses the shared `ForkJoinPool` — only beneficial for large datasets with CPU-heavy, independent operations; for small collections or I/O-bound work, overhead usually makes it *slower*, and shared thread pool contention can affect unrelated code
Prefer immutable collections for shared/read-only data	`List.of(...)`, `Collections.unmodifiableList(...)` prevent accidental mutation bugs and are inherently thread-safe to read
Avoid unnecessary object creation	Reuse objects where sensible (e.g., compiled `Pattern` objects, cached formatters) instead of recreating them on every call

```
// Bad: unknown final size, triggers multiple resizes
List<Integer> list = new ArrayList<>();
for (int i = 0; i < 100000; i++) list.add(i);

// Good: pre-sized, no resize overhead
List<Integer> list2 = new ArrayList<>(100000);
```

Part 18 — Frequently Confused Concepts

⭐⭐⭐⭐⭐

ArrayList vs LinkedList

ArrayList	LinkedList
Array-backed, O(1) random access	Node-backed, O(n) random access
O(n) insert/delete in middle	O(1) insert/delete at known node

HashSet vs TreeSet

HashSet	TreeSet
No order guarantee, O(1) avg ops	Sorted order, O(log n) ops
Allows one null	Disallows null

HashMap vs TreeMap

HashMap	TreeMap
No order, O(1) avg	Sorted by key, O(log n)
Backed by bucket array	Backed by Red-Black Tree

HashMap vs LinkedHashMap

HashMap	LinkedHashMap
No iteration order guarantee	Preserves insertion (or access) order

HashMap vs Hashtable

HashMap	Hashtable
Not synchronized, allows 1 null key	Fully synchronized, no nulls allowed

HashMap vs ConcurrentHashMap

HashMap	ConcurrentHashMap
Not thread-safe	Thread-safe via fine-grained locking
Allows null key	Disallows null key/value

Iterator vs ListIterator

Iterator	ListIterator
Forward-only, any Collection	Bidirectional, List only
Only `remove()`	`remove()`, `set()`, `add()`

Comparable vs Comparator

Comparable	Comparator
`compareTo()`, inside the class	`compare()`, external
One natural ordering	Unlimited custom orderings

Collection vs Collections

Collection	Collections
Interface (List/Set/Queue root)	Utility class of static helper methods

Stream vs Collection

Collection	Stream
Stores data, in-memory data structure	Doesn't store data — a pipeline processing data from a source
Can be iterated multiple times	Consumed once by a terminal operation
Eager	Lazy (until a terminal op runs)

`map` vs `flatMap`

`map`	`flatMap`
1-to-1 transform, can produce nested streams	Transforms + flattens into a single-level stream

`forEach` vs `peek`

`forEach`	`peek`
Terminal operation	Intermediate operation
Meant to be the pipeline's final action	Meant for debugging/inspection mid-pipeline

`orElse` vs `orElseGet`

`orElse`	`orElseGet`
Argument evaluated eagerly, always	Supplier invoked lazily, only if empty

Record vs POJO

Record	Traditional POJO (class)
Implicitly final, immutable	Mutable by default
Auto-generates constructor/accessors/equals/hashCode/toString	Must hand-write (or use Lombok)
Best for simple, immutable data carriers (DTOs)	Best for objects needing mutable state or inheritance

Part 19 — Real Project Usage

Use Case	Collections/Java 8+ Concepts Applied
Spring Boot / REST APIs	`List<T>` / `Map<K,V>` for request/response bodies; `Optional<T>` for repository lookups (`findById` returns `Optional<Entity>`)
DTO Transformation	Streams `.map()` to convert `List<Entity>` → `List<DTO>` in one line
Caching	`LinkedHashMap` (access-order mode) for LRU caches; `ConcurrentHashMap` for thread-safe application-level caches
Pagination	`stream().skip(offset).limit(pageSize)` patterns (though DB-level pagination is preferred for large datasets)
Sorting	`Comparator` chains (`thenComparing`) for multi-level sort in reports/listings
Filtering	Stream `.filter()` for search/query logic on in-memory collections
Aggregation	`Collectors.groupingBy` + `counting` / `summingInt` for dashboards, reports, analytics
E-commerce	`groupingBy(Order::getStatus)` for order dashboards; `PriorityQueue` for order-processing priority; `TreeMap` for price-range lookups
Banking	`TreeMap` for interest-rate slabs (range lookups via `floorKey`); immutable `List.of()` for transaction audit trails

```
// Classic Spring Boot pattern combining several Part topics at once:
List<UserDTO> activeUserDTOs = userRepository.findAll().stream()
    .filter(User::isActive)
    .sorted(Comparator.comparing(User::getCreatedAt).reversed())
    .map(user -> new UserDTO(user.getName(), user.getEmail()))
    .collect(Collectors.toList());

Optional<User> user = userRepository.findById(id);
User result = user.orElseThrow(() -> new UserNotFoundException(id));
```

Part 20 — Placement Focus Summary

Topic	Importance	Why Companies Ask
HashMap internals	⭐⭐⭐⭐⭐	THE most-asked Java topic — tests real depth beyond API usage
ArrayList vs LinkedList	⭐⭐⭐⭐⭐	Foundational data structure trade-off understanding
Streams (map/filter/collect)	⭐⭐⭐⭐⭐	Modern Java is written this way — near-guaranteed in coding rounds
Collectors (groupingBy)	⭐⭐⭐⭐⭐	Extremely common coding-round exercise (word frequency, grouping)
Optional	⭐⭐⭐⭐⭐	Tests modern null-safety awareness
Comparable vs Comparator	⭐⭐⭐⭐	Common in sorting-based coding questions
Fail-fast vs Fail-safe	⭐⭐⭐⭐	Tests understanding of concurrent modification pitfalls
ConcurrentHashMap	⭐⭐⭐⭐	Relevant for any backend/multithreaded discussion
Records & Pattern Matching	⭐⭐⭐⭐	Signals current Java 17/21 knowledge
Lambda "effectively final"	⭐⭐⭐⭐	Classic gotcha that separates surface vs real understanding

Part 21 — Coding Practice

(Problem prompts for self-practice — solve using Streams/Collections concepts from this handbook.)

10 Stream Problems

1. Find the sum of all even numbers in a `List<Integer>`.
2. Convert a `List<String>` to a comma-separated `String` (no trailing comma).
3. Find the longest string in a `List<String>`.
4. Count the frequency of each word in a `List<String>` using `groupingBy` + `counting`.
5. Group a `List<Employee>` by department into `Map<String, List<Employee>>`.
6. Find the second-highest number in a `List<Integer>` using Streams.
7. Flatten a `List<List<Integer>>` into a single sorted, distinct `List<Integer>`.
8. Partition a `List<Integer>` into even and odd using `partitioningBy`.
9. Find the average salary per department using `groupingBy` + `averagingDouble`.
10. Convert a `List<Employee>` into a `Map<String, Double>` of name → salary using `toMap`.

10 Collection Problems

1. Remove duplicates from a `List<Integer>` while preserving insertion order.
2. Find the intersection of two `List<Integer>`s.
3. Implement an LRU cache using `LinkedHashMap`.

4. Sort a `Map<String, Integer>` by its values in descending order.
5. Find the top-3 highest values in a `List<Integer>` using a `PriorityQueue`.
6. Check if two `List<Integer>`s contain the same elements regardless of order.
7. Implement a stack-based valid-parentheses checker using `ArrayDeque`.
8. Merge two sorted `List<Integer>`s into one sorted list without using `Collections.sort()` afterward.
9. Find all pairs in a `List<Integer>` that sum to a target value using a `HashSet`.
10. Reverse a `LinkedList` in place (no extra data structure).

5 HashMap Debugging Questions

1. A custom key class overrides `equals()` but not `hashCode()` — why does `map.get()` fail to find an entry that was clearly `put()` earlier?
2. A mutable object is used as a HashMap key, mutated after insertion, then `get()` fails — explain why and how to fix it.
3. Iterating a `HashMap` and calling `map.remove()` directly inside a for-each throws an exception — why, and what's the fix?
4. Two `Integer` keys with the same numeric value inserted separately somehow result in **one** entry, not two — is this a bug? Explain.
5. A `HashMap<String, List<Integer>>` accumulates values via `map.get(key).add(x)`, but throws `NullPointerException` on first insert for a new key — what's the fix? (Hint: `computeIfAbsent`.)

5 Comparator Problems

1. Sort a `List<Employee>` by salary ascending, then by name descending for ties.
2. Sort a `List<String>` by length, then alphabetically for equal lengths.
3. Sort a `Map<String, Integer>`'s entries by value using `Comparator.comparing(Map.Entry::getValue)`.
4. Write a `Comparator` that sorts `null`s last, using `Comparator.nullsLast()`.
5. Sort a `List<Employee>` in reverse natural order using `Comparator.reverseOrder()`.

5 Optional Problems

1. Write a method returning `Optional<User>` from a lookup, and safely chain `.map()` to extract the user's email or a default.
2. Refactor a null-check-heavy method to use `Optional.ofNullable()` + `orElseThrow()`.
3. Use `Optional.flatMap()` to safely navigate a chain: `User → Address → City` where any step might be absent.
4. Explain why `optional.orElse(fetchFromDB())` is a performance bug and fix it.
5. Convert a `List<Optional<String>>` into a `List<String>` containing only the present values.

5 Output Prediction Questions

1. `Integer a=127,b=127; System.out.println(a==b);` then same with `200` — predict both outputs.
2. Predict the output of iterating a `HashSet<String>` after adding `"b","a","c"` — is order guaranteed?
3. `list.stream().peek(System.out::println).count();` — does `peek` actually print anything? (Hint: some JVM versions optimize this away.)
4. Predict what happens when you `add()` to a list obtained from `List.of(1,2,3)`.
5. Predict the output of `Stream.of(1,2,3).map(n -> n*2).findFirst().get()` and explain why the whole stream isn't processed.

Part 22 — Final Revision

Collections Cheat Sheet

- **List** = ordered, duplicates allowed. `ArrayList` (fast random access) vs `LinkedList` (fast end insert/delete).
- **Set** = unique elements. `HashSet` (no order) vs `LinkedHashSet` (insertion order) vs `TreeSet` (sorted).
- **Map** = key-value pairs, **not** a `Collection`. `HashMap` (no order) vs `LinkedHashMap` (order) vs `TreeMap` (sorted).
- **Queue/Deque** = `PriorityQueue` (heap-ordered) vs `ArrayDeque` (modern stack/queue choice).
- HashMap: hash → bucket → chain/tree; load factor 0.75; treeifies at 8+ collisions per bucket.
- Fail-fast iterators throw `ConcurrentModificationException`; fail-safe ones (Concurrent collections) don't.

Stream Cheat Sheet

- Streams are **lazy** until a terminal operation runs; can be consumed **only once**.
- `map` = 1-to-1 transform; `flatMap` = transform + flatten nested streams.
- `filter` / `distinct` / `sorted` / `limit` / `skip` / `peek` = intermediate (lazy).
- `collect` / `forEach` / `reduce` / `count` / `findFirst` / `anyMatch` = terminal (eager).
- `anyMatch` / `allMatch` / `noneMatch` / `findFirst` / `limit` can **short-circuit** — stop early.
- `groupingBy` = multiple groups; `partitioningBy` = exactly true/false groups.

Optional Cheat Sheet

- Use as a **return type only** — never as a field or parameter type.
- `orElse()` evaluates eagerly (always); `orElseGet()` evaluates lazily (only if empty).
- Chain safely with `.map()` / `.flatMap()` instead of calling `.get()` directly.
- `Optional.ofNullable()` for values that might be null; `Optional.of()` throws if given null.

Complexity Table (Master Reference)

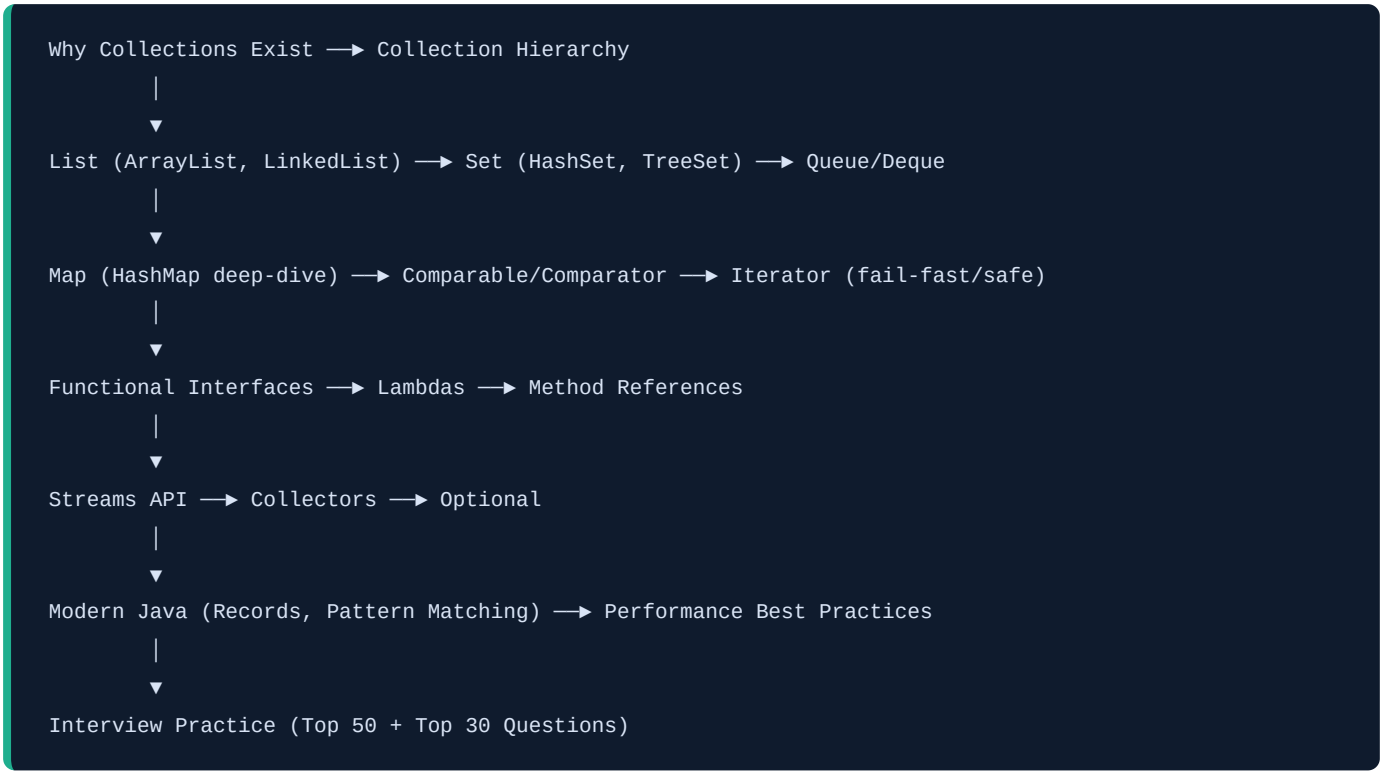
Structure	Access	Search	Insert	Delete
ArrayList	O(1)	O(n)	O(1) amortized (end) / O(n) (middle)	O(n)
LinkedList	O(n)	O(n)	O(1) (ends)	O(1) (with reference)
HashSet/HashMap	—	O(1) avg / O(log n) worst	O(1) avg	O(1) avg
TreeSet/TreeMap	—	O(log n)	O(log n)	O(log n)
PriorityQueue	—	O(n)	O(log n)	O(log n)
ArrayDeque	O(1) (ends)	O(n)	O(1) (ends)	O(1) (ends)

Memory Tricks

- **HashMap load factor:** "0.75 — three-quarters full triggers the growth spurt"

- **PECS:** "Producer Extends, Consumer Super" (from Handbook 2, still applies to generic collection methods)
- **Treeification threshold:** "8 strikes and the bucket becomes a tree"
- **orElse vs orElseGet:** "orElse ALWAYS runs, orElseGET only runs when GEtting is actually needed"

Topic Dependency Map



20-Minute Last-Minute Revision Checklist

- [ ] Can you explain what happens step-by-step inside `hashMap.put()`?
- [ ] Can you explain treeification and the 0.75 load factor?
- [ ] Can you explain ArrayList vs LinkedList time complexities?
- [ ] Can you explain fail-fast vs fail-safe iterators?
- [ ] Can you explain lazy evaluation and short-circuiting in Streams?
- [ ] Can you explain map vs flatMap with an example?
- [ ] Can you explain groupingBy vs partitioningBy?
- [ ] Can you explain orElse vs orElseGet?
- [ ] Can you explain "effectively final" for lambda captures?
- [ ] Can you write a word-frequency counter using Streams from memory?

Top 50 Collections Interview Questions

1. **Why is Map not a subtype of Collection?** — Map deals in key-value pairs, not single elements; it's a separate hierarchy.
2. **ArrayList vs LinkedList?** — ArrayList: O(1) random access, O(n) middle insert. LinkedList: O(n) access, O(1) end insert/delete.
3. **What is ArrayList's growth factor?** — 1.5x current capacity when full.
4. **Why does removing by index in a for-loop skip elements?** — Indices shift after removal, but the loop counter still increments.
5. **How does HashSet ensure uniqueness?** — Backed by HashMap; uses hashCode() + equals() to detect duplicates.
6. **Why can't TreeSet contain null?** — Comparisons for sorting would throw NullPointerException.

7. **What is treeification?** — Converting a bucket's linked list to a red-black tree at 8+ collisions, bounding lookup to O(log n).
8. **Walk through HashMap.put() internals.** — Compute hash, find bucket index, check for collision via equals(), insert or update.
9. **What happens if a mutable HashMap key is changed after insertion?** — The entry becomes unreachable — old bucket holds it, but lookups compute a new hash pointing elsewhere.
10. **Why is HashMap's default load factor 0.75?** — Balances memory usage against collision frequency.
11. **HashMap vs Hashtable?** — HashMap is unsynchronized, allows one null key; Hashtable is fully synchronized, no nulls.
12. **HashMap vs ConcurrentHashMap?** — ConcurrentHashMap uses fine-grained locking for better concurrent throughput; both disallow null in the concurrent case.
13. **What does LinkedHashMap add over HashMap?** — Preserves insertion (or access) order via an internal linked list.
14. **How would you build an LRU cache using LinkedHashMap?** — Use access-order mode + override `removeEldestEntry()`.
15. **TreeMap's backing structure?** — Red-Black Tree, giving O(log n) operations and sorted iteration.
16. **What is a PriorityQueue backed by?** — A binary heap stored in an array.
17. **Why prefer ArrayDeque over Stack/LinkedList?** — Better performance, no unnecessary synchronization, no per-node overhead.
18. **Iterator vs ListIterator?** — Iterator is forward-only for any Collection; ListIterator is bidirectional, List-only, and supports add()/set().
19. **What causes ConcurrentModificationException?** — Structural modification of a collection during fail-fast iteration.
20. **Fail-fast vs fail-safe?** — Fail-fast throws on concurrent structural change (via modCount); fail-safe iterates a snapshot or tolerates changes.
21. **Comparable vs Comparator?** — Comparable defines one natural ordering inside the class; Comparator defines external, unlimited custom orderings.
22. **How do you sort by multiple fields?** — `Comparator.comparing(...).thenComparing(...)`.
23. **Collection vs Collections?** — Collection is an interface; Collections is a static utility class.
24. **What does Collections.unmodifiableList() do?** — Returns a read-only view that throws on mutation attempts.
25. **How do you make a list thread-safe?** — `Collections.synchronizedList()` (coarse lock) or `CopyOnWriteArrayList` (for read-heavy use).
26. **What is the time complexity of binarySearch, and its precondition?** — O(log n); the list/array must already be sorted.
27. **Why might two equal HashMap keys end up in different buckets?** — Inconsistent/missing hashCode() override relative to equals().
28. **What's the danger of a poorly distributed hashCode()?** — Many collisions degrade a bucket toward O(n) lookup (or O(log n) post-treeification).
29. **Why does HashSet allow only one null?** — It's backed by HashMap, which permits exactly one null key.
30. **What is the default initial capacity of a HashMap?** — 16.
31. **When does a HashMap resize?** — When size exceeds capacity × load factor (default 0.75).
32. **How would you make a HashMap iteration-order-safe for display purposes?** — Use LinkedHashMap or sort entries explicitly (e.g., into a TreeMap).
33. **What's the risk of using a mutable object as a Map key?** — If mutated after insertion, the entry can become unreachable.
34. **NavigableMap's floorKey vs ceilingKey?** — floorKey returns the largest key ≤ given key; ceilingKey returns the smallest key ≥ given key.
35. **Why is Vector considered legacy?** — Its synchronization is coarse and usually unnecessary overhead; ArrayList + explicit sync (or concurrent collections) is preferred.
36. **What's the difference between poll() and remove() on a Queue?** — poll() returns null on empty queue; remove() throws an exception.
37. **How is a Deque used as both a stack and a queue?** — push/pop for stack (LIFO); offer/poll for queue (FIFO).
38. **Why is ArrayList generally preferred over LinkedList in practice?** — Better cache locality and lower per-element memory overhead, despite LinkedList's theoretical end-operation advantage.
39. **What happens when you call Collections.sort() on a list of custom objects with no Comparable/Comparator?** — Compile error (or runtime ClassCastException with raw types) — the objects must implement Comparable or a Comparator must be

supplied.

40. **How would you find duplicate elements in a List efficiently?** — Use a HashSet to track seen elements while iterating, O(n) overall.
41. **What is the purpose of the initial capacity constructor argument in HashMap/ArrayList?** — Pre-sizing to avoid repeated resize/rehash operations.
42. **Why does iterating a HashMap not guarantee order even across two runs with the same insertions?** — Order depends on hash values and internal bucket layout/resizing history, not insertion sequence.
43. **What's a real-world use case for TreeMap over HashMap?** — Range queries, like finding all entries within a price/date range via `headMap` / `tailMap` / `subMap`.
44. **Explain the difference between Set and List conceptually.** — List preserves order and allows duplicates; Set enforces uniqueness with no positional indexing.
45. **Why is EnumMap generally faster than HashMap for enum keys?** — Backed by a simple array indexed by ordinal, avoiding hashing entirely.
46. **What's the significance of the "capacity is always a power of two" detail in HashMap?** — Enables the fast `hash & (capacity-1)` bucket index calculation instead of a slower modulo operation.
47. **How would you safely remove elements while iterating a List?** — Use `Iterator.remove()`, or iterate backwards with an index-based loop.
48. **What's the difference between `size()` and `capacity()` conceptually for ArrayList?** — `size()` is the number of actual elements; capacity is the underlying array's allocated length (capacity is not directly exposed by ArrayList's public API).
49. **Can a TreeMap have a custom sort order?** — Yes, by supplying a `Comparator` in its constructor instead of relying on natural ordering.
50. **Why might ConcurrentHashMap still show a slightly stale count during concurrent modification?** — Its weakly consistent iterators/size() trade off strict accuracy for high concurrent throughput, rather than locking the whole map.

Top 30 Stream Interview Questions

1. **What is a Stream?** — A lazy pipeline for processing a sequence of elements functionally; it doesn't store data itself.
2. **Intermediate vs terminal operations?** — Intermediate ops are lazy and return a Stream; terminal ops are eager and trigger execution.
3. **Why are Streams lazy?** — To allow operation fusion (single pass) and short-circuiting, improving performance.
4. **Can a Stream be reused?** — No — consumed once by a terminal operation; reuse throws IllegalStateException.
5. **map vs flatMap?** — map is 1-to-1; flatMap transforms and flattens nested streams into one.
6. **What does short-circuiting mean?** — Certain operations (findFirst, anyMatch, limit) can stop processing early without touching the whole source.
7. **How do you count word frequency with Streams?** — `groupingBy(w -> w, counting())`.
8. **groupingBy vs partitioningBy?** — groupingBy makes arbitrary groups; partitioningBy always makes exactly two (true/false).
9. **How do you flatten a List>?** — `.flatMap(List::stream)`.
10. **How do you convert a List to a Map with Streams?** — `.collect(Collectors.toMap(keyFn, valueFn))`.
11. **forEach vs peek?** — forEach is terminal (the pipeline's final action); peek is intermediate (debugging/inspection).
12. **What does reduce() do?** — Combines all stream elements into a single result using an accumulator function.
13. **How do you get distinct elements in a Stream?** — `.distinct()`, based on equals()/hashCode().
14. **What is the difference between findFirst and findAny?** — findFirst returns the first encountered element (ordered); findAny may return any matching element, useful for parallel streams (no order guarantee, potentially faster).
15. **How would you sort a Stream by multiple fields?** — `.sorted(Comparator.comparing(...).thenComparing(...))`.
16. **What does Collectors.joining() do?** — Concatenates Stream elements, optionally with a delimiter/prefix/suffix.
17. **How would you compute an average with Streams?** — `.mapToInt(...).average()` or a `summarizingInt()` collector.
18. **What is IntStream and why use it?** — A primitive-specialized stream avoiding autoboxing overhead for numeric operations.

19. **When should you use a parallel stream?** — Only for large datasets with CPU-heavy, independent operations — not for small collections or I/O-bound work.
20. **What thread pool do parallel streams use by default?** — The shared common `ForkJoinPool`.
21. **How do you filter then transform then collect to a List?** — `.filter(...).map(...).collect(Collectors.toList())`.
22. **What does collectingAndThen do?** — Applies an extra finishing transformation after a collector completes (e.g., wrapping in an unmodifiable list).
23. **How would you find the max element in a Stream?** — `.max(Comparator.naturalOrder())` (returns an Optional).
24. **Why does .max()/.min() return an Optional?** — The stream could be empty, so there might be no max/min to return.
25. **How do you generate an infinite stream and safely limit it?** — `Stream.iterate(seed, next)` combined with `.limit(n)` or a short-circuiting terminal operation.
26. **What does Stream.generate() do?** — Produces an infinite stream from a Supplier — must be limited to avoid running forever.
27. **How do you skip the first N elements of a Stream?** — `.skip(n)`.
28. **What happens if you call .sorted() on an infinite stream without limiting first?** — It never terminates — sorted needs to see the whole stream before producing output, incompatible with unbounded sources unless bounded first.
29. **How would you check if all elements match a condition?** — `.allMatch(predicate)` — short-circuits on the first failure.
30. **How do you convert a Stream back to an array?** — `.toArray()` or `.toArray(String[]::new)` for a typed array.

Key Takeaways

- **HashMap internals are the single highest-leverage topic** in this handbook — know the put()/get() flow, hashing, collisions, treeification, and load factor cold.
- **Streams are lazy** — nothing runs until a terminal operation, which is why operation order and short-circuiting matter.
- **Optional exists to make "maybe absent" explicit** — use it as a return type, chain with `map` / `flatMap`, and prefer `orElseGet` for expensive defaults.
- **Know your Big-O** for every collection's core operations — it's the fastest way to justify a design choice in an interview.
- **Records and pattern matching are now expected knowledge** — modern Java (17/21) shows up in interviews as often as Java 8.

End of Handbook 3. This completes the core "Java Placement Master Series" — revisit Handbooks 1 and 2 alongside this one for full-spectrum placement readiness.